Number Systems, Representation, and Arithmetic Hardware

Lecture Notes — Week 2

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

2.1 Introduction

Week 1 established the performance equation, CPU time $= \mathrm{IC} \times \mathrm{CPI}/f$, and with it the assumption that we can count instructions and measure CPI. But a CPU cannot execute anything until the **data** it works on is represented in bits, and arithmetic is implemented in hardware. This chapter is about what the bits mean and how hardware adds them. It answers four questions that form one thread: how we write numbers in binary, octal, and hex and convert between bases (Section 2.2); how hardware represents negative integers in two's complement (Section 2.3); when arithmetic fails (overflow) and how fractions are represented in fixed point (Section 2.4); and how we build a circuit that actually adds (Section 2.5).

Throughout the chapter we thread a single worked example: adding $+7 + (-3)$ in binary, bit by bit, then building the circuit that does it, and checking the result against the range of the format. The thread is closed in Section 2.6 when the same addition is run through a ripple-carry adder in hardware.

2.2 Number Systems

2.2.1 Motivation: One Value, Many Alphabets

The decimal number 13 can be written with two symbols, or eight, or sixteen — each is the same value in a different alphabet. Decimal $13 = 1 \times 10^1 + 3 \times 10^0$. Binary $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$. Octal 15_8 and hex D_{16} are each a shorthand for groups of bits. Hardware stores *bits*; the base is just how we write them down.

Definition (Positional Number System). *In base r, a number with digits $d_{n-1} \ldots d_1 d_0$ has value*

$$d_{n-1} r^{n-1} + \cdots + d_1 r^1 + d_0 r^0.$$

The base is written as a subscript: 1101_2, 15_8, D_{16}.

The common bases are $r = 2$ binary (digits 0,1), $r = 8$ octal (0–7), $r = 10$ decimal, and $r = 16$ hex (0–9, A–F). **Conversion** works by repeated division: dividing by r repeatedly yields the digits from least to most significant. Hex and octal are just binary in **groups**: one hex digit is four bits, one octal digit is three bits.

Example 2.2.1 (Decimal to binary, and back)**.** *Convert* 53_{10} *to binary by repeated division by 2:* $53/2 = 26$ *r* 1*;* $26/2 = 13$ *r* 0*;* $13/2 = 6$ *r* 1*;* $6/2 = 3$ *r* 0*;* $3/2 = 1$ *r* 1*;* $1/2 = 0$ *r* 1*. Reading the remainders bottom-up gives* 110101_2*. Check by expanding:* $1{\times}32 + 1{\times}16 + 0{\times}8 + 1{\times}4 + 0{\times}2 + 1{\times}1 = 53$*. Binary* $\leftrightarrow$ *hex is trivial by grouping:* $1101\,0101_2 = D5_{16}$.

2.2.2 Socratic Check: How Many Bits?

How many bits do you need to represent n distinct values? The answer is $\lceil \log_2 n \rceil$ bits. With k bits, the largest unsigned integer is $2^k - 1$: 8 bits give 0 to 255, 16 bits give 0 to 65535. This ceiling matters the moment we talk about **overflow**.

2.3 Signed Representation

2.3.1 Three Candidate Schemes for Negatives

Given n bits, how do we encode a sign? Three schemes compete. Sign-magnitude uses one sign bit plus $n-1$ magnitude bits — simple, but it has two zeros (+0 and −0) and awkward addition. One's complement negates by flipping all bits — still two zeros, and addition needs an end-around carry. Two's complement negates by flipping all bits and adding 1 — it has one zero, and *addition just works*: $-x$ is defined so that $x + (-x) = 0$ in the same bit width, with the carry-out simply discarded. The ALU needs only an adder.

Definition (Two's Complement). *The two's complement of an n-bit number x is $-x \equiv 2^n - x \pmod{2^n}$, equivalently "flip all bits and add 1." The representable range is $-2^{n-1} \le x \le 2^{n-1} - 1$; the sign bit is the MSB (0 for non-negative, 1 for negative). Adding a number to its two's complement gives all zeros — the "self-cancelling" property that makes subtraction free (P&H, Sec. 3.2, p.188; Hamacher Ch. 2) [3, 1].*

Example 2.3.1 (+7 and −3 in 8-bit two's complement)**.** *$+7 = 0000\,0111_2$. To form -3: flip $0000\,0011 \to 1111\,1100$, add 1, giving $1111\,1101_2$. Check by adding: $0000\,0111 + 1111\,1101 = 1\,0000\,0100$; discard the carry-out (the 9th bit) to get $0000\,0100 = +4$, i.e. $7 + (-3) = 4$. Negative numbers are ordinary bits, and addition needs no special subtraction hardware — just discard the carry-out.*

2.3.2 Socratic Check: Negating Without a Calculator

−5 in 8-bit two's complement is 1111 1011. Its own negation: flip to 0000 0100, add 1, giving 0000 0101 = +5. Negating twice returns the original — two's complement is an **involution**: $-(-x) = x$. This self-inverse property is why negation is easy to implement and verify.

2.4 Overflow and Fixed-Point

2.4.1 What Is Overflow?

Overflow occurs when the true result of an arithmetic operation cannot be represented in the format's range. For unsigned numbers, a carry out of the MSB is overflow. For two's complement, overflow is *not* the same as carry-out: a carry out of the MSB does not always mean overflow (recall $7 + (-3)$ above, which carried out yet was correct). The correct condition is that overflow occurs when the sign of the result is *wrong* — adding two positives gives a negative, or two negatives give a positive (P&H, Sec. 3.2, p.188) [3].

Example 2.4.1 (Overflow vs. carry in 8-bit two's complement)**.** *The 8-bit two's complement range is −128 to +127.*

- *+100 + +50: $0110\,0100 + 0011\,0010 = 1001\,0110$, which reads as −106 — wrong sign, so **overflow**.*

- $+127 + 1$*:* $0111\,1111 + 1 = 1000\,0000 = -128$ — *again wrong sign, overflow.*
- $-1 + (-1)$*:* $1111\,1111 + 1111\,1111 = 1\,1111\,1110$*; discard the carry to get* $1111\,1110 = -2$ — *correct.*

The rule of thumb: overflow happens exactly when the two operands share a sign bit and the result's sign bit differs. Carry-out alone is not the test.

2.4.2 Fixed-Point Representation

A **fixed-point** number has an agreed position for the "binary point":

$$b_{k-1} \dots b_0 . b_{-1} b_{-2} \dots b_{-m}, \qquad \text{value} = \sum_i b_i 2^i.$$

For example, `Q8.8` means 8 integer bits and 8 fraction bits, where each fractional bit is $2^{-1}, 2^{-2}, \dots$. Thus $0.5 = 0.1_2$, $0.25 = 0.01_2$, $0.75 = 0.11_2$. Fixed point is simple and fast — add/sub work on the same integer hardware — but the fixed point limits range and precision, which is why Week 3 moves to floating point (Hamacher Ch. 2; Mano Ch. 1) [1, 2].

Example 2.4.2 (A fixed-point addition)**.** *Add* $3.5 + 0.75$ *in Q8.8.* $3.5 = 0000\,0011.1000\,0000$*;* $0.75 = 0000\,0000.1100\,0000$*. Adding bitwise:* $0000\,0100.0100\,0000$*, which reads as* 4.25*. The integer ALU adds fixed-point values unchanged — the programmer just agrees where the point is.*

2.4.3 Socratic Check: When Does Fixed Point Fail?

0.1 in binary is a **repeating** fraction ($0.000110011\dots_2$) — it does not terminate. Fixed point must **truncate**, so 0.1 is stored approximately, and repeated arithmetic accumulates error. This is the fundamental limitation that floating point (Week 3) manages but does not eliminate.

2.5 Adder Hardware

2.5.1 Motivation: From Addition to a Circuit

A human adds two numbers one column at a time, carrying. The smallest circuit that adds two *single bits* plus a carry-in is the **full adder**; chaining n of them builds a multi-bit adder. The **half adder** (HA) has two inputs A, B and outputs sum S and carry C; the **full adder** (FA) has three inputs A, B, C_{in} and outputs sum S and carry C_{out}.

Definition (Half Adder and Full Adder).

$$\textit{HA: } S = A \oplus B, \quad C = A \cdot B; \qquad \textit{FA: } S = A \oplus B \oplus C_{in}, \quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B).$$

The full adder's truth table (Table 1) has the pattern: count the 1s among (A, B, C_{in}) — if odd, $S = 1$; if two or more, $C_{out} = 1$.

2.5.2 The Ripple-Carry Adder

The **ripple-carry adder** chains n full adders so that each FA's C_{out} feeds the next FA's C_{in} (Figure 2.1). The carry **ripples** from the least-significant to the most-significant bit — hence the name — and the final C_{out} is the unsigned overflow flag.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full adder truth table.

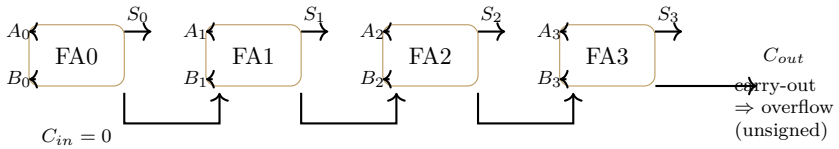

Figure 2.1: The ripple-carry adder: full adders chained by their carry signals.

Example 2.5.1 (Adding $7 + (-3)$ in hardware)**.** $+7 = 0111$, $-3 = 1101$ *(4-bit two's complement). Add with a 4-bit ripple-carry adder,* $C_{in} = 0$*: bit 0:* $1 + 1 = 0$ *carry 1; bit 1:* $1 + 0 + 1 = 0$ *carry 1; bit 2:* $1 + 1 + 1 = 1$ *carry 1; bit 3:* $0 + 1 + 1 = 0$ *carry 1. Result* $0100 = +4$*, carry-out* $= 1$ *(discarded). One adder, same circuit, handles both signed and unsigned — the interpretation is the programmer's choice.*

2.5.3 Socratic Check: Adder and Subtraction, One Circuit

The ALU has only an adder. To compute $A - B$, one extra step suffices: subtract B's two's complement, $A - B = A + (-B)$. Since $-B$ is "flip B + add 1," this is implementable with a MUX (choose B or $\bar{B}$) plus a forced carry-in of 1. Subtraction is therefore **free**: one adder plus a MUX, no separate subtractor needed (P&H, Sec. 3.2, p.188) [3].

2.6 Synthesis and Summary

2.6.1 The Thread, Closed

The chapter's running thread — adding $+7 + (-3)$ — used every tool. We represented both operands in two's complement (0111 and 1101), recognized that addition needs no special subtraction hardware, ran the bits through a ripple-carry adder, discarded the carry-out, and checked the sign to confirm no overflow. The result, $0100 = +4$, is the same number a human would compute — but now it is the output of a specific circuit.

2.6.2 Bridge to Week 3

Fixed point handles fractions but with a fixed range and precision. Real numbers span enormous ranges (from 10^{-300} to 10^{300} in science); one fixed point cannot cover them. Week 3 builds **floating point**: the IEEE 754 representation — sign, exponent, mantissa — trading a little precision for a huge range, plus the normalization and the arithmetic.

2.6.3 Summary

Number systems: value $= \sum d_i r^i$; convert by repeated division; hex/octal group bits. **Two's complement:** $-2^{n-1} \leq x \leq 2^{n-1} - 1$; negate by flip + 1; addition needs no special subtraction hardware. **Overflow:** carry-out $\neq$ overflow; a wrong result sign is the test. **Fixed point:** an agreed binary point; simple but limited range and precision. **Adder:** HA/FA equations; the ripple-carry chain; $A - B = A + (-B)$.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.

[3] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.